nC

DOCUMENTACIÓN DEL LENGUAJE

nC Arquitectura y bootstrap

Contratos del compilador, memoria, ABI, materialización y alcance de la reconstrucción nativa V81.

Edición de bootstrap V81
24 de octubre de 2026
github.com/Arizt00/nc

01 El contrato entre niveles
02 Parser AST semántica e IR
03 Máquina y control de flujo
04 Memoria y representación
05 Materialización ABI y enlace
06 Qué demuestra el bootstrap V81
07 Reproducir y revisar evidencia
08 Publicación y contribuciones

El contrato entre niveles

nC organiza distintas formas de expresar un programa dentro de una continuidad semántica. La fuente normal, la raíz basal, la capa de máquina y las primitivas tienen responsabilidades diferentes. Una operación debe conservar la información necesaria al atravesar esos niveles.

El repositorio contiene la implementación Rust retenida y el trabajo nativo en nC. `crates/` reúne los componentes de la primera; `tools/ncc/nc/src/` contiene fuentes del compilador nativo. `runtime/nc/` conserva cuerpos y proveedores del runtime. Las herramientas de construcción y los contratos bajo `bootstrap/` relacionan generaciones y entradas.

Fase	Responsabilidad
Lexer y parser	Tokenización y reconocimiento de la estructura sintáctica
AST	Identidad de declaraciones, tipos, relaciones y superficies
Semántica	Resolución de símbolos y validación de tipos, aridad y flujo
IR	Representación del programa analizado para lowering y transformaciones
MCT y backend	Materialización conforme al layout, la ISA y la ABI del destino
Enlace	Resolución de símbolos, relocaciones y dependencias de la imagen
Ejecución	Comprobación del resultado y de los efectos observables

La tabla es un mapa de responsabilidades, no una promesa de que cada ruta atraviese exactamente las mismas funciones. Algunas superficies tienen entradas y validaciones propias. Un cambio correcto explica qué ruta modifica y qué información conserva.

La documentación de diseño y la evidencia de una versión cumplen funciones distintas. La primera define el contrato; la segunda demuestra resultados para unas entradas, un compilador y un entorno concretos. Las dos deben poder cotejarse con el código.

Parser AST semántica e IR

El parser actual reconoce imports compactos como `use <io:output>;`, contenedores con nombre, secciones y funciones agrupadas. Las pruebas de Material V2 en `crates/ncc-parser/src/lib.rs` fijan comportamientos de esa superficie. La denominación histórica de una prueba no debe usarse para deducir una gramática distinta de la que comprueba.

El AST debe mantener la identidad de los elementos que luego necesita el análisis: lugares, funciones, tipos, visibilidad, flujo, contratos de layout y superficies materiales. Si un dato ya tiene una representación tipada, sustituirlo por una cadena de texto trasladaría errores a fases posteriores.

La semántica comprueba las relaciones aceptadas por el lenguaje. Una firma debe tener aridad y tipos coherentes; las referencias deben resolver al símbolo correcto; las operaciones de control deben respetar su contexto. Un diagnóstico consolidado es parte del comportamiento observable de las herramientas.

El IR conserva el resultado del análisis de forma utilizable por las fases siguientes. No basta con que un nodo exista en el AST: hay que comprobar que su significado sobreviva al lowering y a las transformaciones. Las pruebas deben inspeccionar las propiedades que podrían perderse.

Un cambio de superficie debería incluir un caso positivo, otro negativo y una observación del IR cuando el contrato pueda alterarse en la traducción. Si llega a generación nativa, debe comprobar además el artefacto y, cuando el entorno lo permita, su ejecución.

Las pruebas deben cubrir los puntos donde puede perderse información: resolución del destino de un salto, compatibilidad de firmas y conservación del orden de evaluación. Esas propiedades necesitan casos observables y expectativas independientes de la implementación.

Máquina y control de flujo

`[@machine]` expresa instrucciones con estructura tipada. Los operandos, registros, etiquetas y finales deben ser coherentes desde el parser hasta el IR. La prioridad del proyecto exige mantener esa relación cuando se amplía la materialización.

Una etiqueta identifica una posición o bloque conforme al modelo admitido. Un salto enlaza caminos de control. Las condiciones de salto conservan el significado de los flags u operaciones que consumen. Al emitir instrucciones reales, no puede perderse la dependencia que hacía correcta la rama en la representación anterior.

Los terminadores delimitan qué puede ejecutarse después de un bloque. Introducir una instrucción tras un final sin definir cómo se alcanza cambia el grafo. Del mismo modo, aceptar un bloque sin el cierre que necesita su contrato desplaza una inconsistencia hacia el backend.

El archivo `checked_add_overflow_machine.n` permite comprobar una rama relacionada con overflow sin presentar el overflow como una protección automática de toda la aritmética del lenguaje. El caso expresa su propia decisión de control.

Los módulos `.nm` de objeto usan la CLI de máquina. En el Stage0 retenido, `machine emit-ir` es la entrada apropiada para `object_elf64.nm`. La ruta general `emit-ir` no debe anunciarse como intercambiable con ella.

Una extensión de ISA exige identificar operaciones, tipos de operandos, restricciones, emisión y pruebas. La presencia de un nombre de destino o de una tabla de instrucciones no demuestra ejecución en hardware. Esa afirmación necesita evidencia separada del perfil correspondiente.

Memoria y representación

La fuente normal mantiene un contrato manual/raw. El compilador no introduce ownership, validación automática de vida de punteros, comprobaciones generales de límites ni liberación oculta. Las propiedades de tipado deben describirse sin convertirlas en garantías de seguridad de memoria.

La representación incluye tamaño, alineación, offsets, padding y ABI. Un tipo agregado puede tener una representación válida y, aun así, requerir reglas distintas al pasarse como argumento o devolverse desde una función. El layout del dato y el protocolo de llamada deben comprobarse juntos.

V81 materializa alineaciones superiores a 16 bytes en el perfil nativo x86-64 de Windows y Linux/WSL. La evidencia recorre potencias de dos entre 32 bytes y 1 MiB. COFF representa como máximo 8192 bytes de alineación de sección en ese trabajo; PE directo y ELF admiten alineaciones mayores dentro del perfil documentado.

Una solicitud que no cabe en el formato debe rechazarse. Truncarla silenciosamente conservaría la apariencia de éxito y cambiaría la dirección real que observa el programa. El verificador de materiales comprueba rangos, potencias de dos y offsets antes de consumirlos.

La pila también tiene un límite material. Las pruebas grandes de Windows declaran una reserva de 16 MiB y las de ABI Linux ajustan el límite de pila según la receta. El emisor toca páginas de pila en el perfil descrito; esto no añade un heap implícito ni elimina los límites del sistema.

Las pruebas de layout en diez perfiles son cálculos de representación. La ejecución física documentada en V81 se refiere a los anfitriones x86-64 indicados. Mantener separadas esas afirmaciones evita confundir portabilidad de un contrato con cierre de todas sus implementaciones.

Materialización ABI y enlace

MCT participa en el camino hacia objetos e imágenes. El backend debe trasladar las decisiones del programa a bytes y metadatos compatibles con el destino. El enlace debe resolver símbolos y relocaciones sin depender por accidente de artefactos retenidos de una construcción anterior.

V81 modifica autoridades nC como `native_body.n`, `mct.n`, `mct_object.n`, `mct_link.n` y `mct_image_elf.n`. Sus responsabilidades incluyen intervalos de pila, temporales, argumentos, retornos, alineación de símbolos y la alineación respecto a direcciones virtuales finales.

Los detalles de ABI importan incluso cuando el programa fuente es el mismo. Los agregados pueden viajar por referencia en Win64 o en un área realineada de argumentos en SysV. Las pruebas incluyen llamadas directas e indirectas, retornos agregados, anidamiento y recursión.

El runtime enlazado debe corresponder a los símbolos que el compilador emite. Un programa que pasa análisis pero no encuentra una exportación sigue sin estar completo. El proveedor, sus dependencias y el mecanismo de búsqueda de bibliotecas pertenecen a la reproducción.

La caché conserva materiales derivados. Su identidad debe cambiar cuando cambia el contrato del emisor. V81 rechaza materiales corruptos incluso cuando se recalcula el digest del manifiesto; verificar solo que el hash coincida con un archivo alterado no valida la estructura de ese archivo.

Para revisar una regresión, conserva por separado fuente, IR o material, objeto, imagen y salida del proceso. Comparar únicamente el texto impreso puede ocultar diferencias en layout, enlace o secciones que reaparecen en otro programa.

Qué demuestra el bootstrap V81

El dossier V81 registra cuatro reconstrucciones finales: B2 y C2 en Windows, y B2 y C2 en Linux/WSL. Cada una reconstruye 74 unidades desde un directorio nuevo mediante `project objects --no-incremental`, sin LTO ni objetos de proyecto retenidos.

Las generaciones reconstruyen además sus proveedores: 36 en Windows y 35 en Linux. La identidad de los compiladores B2 y C2 es exacta dentro de cada anfitrión. Los hashes publicados en el dossier son:

```text
Windows x86-64
4ff2300d30fbe99a4c541481e0fab9df284d77c32d1df082084440b9718854c9

Linux x86-64
e3d6fc744f0768d19a3f44ca93eddcb90cc3f89caee536f68f8f5ce486eae1e7
```

La igualdad es por sistema: no se afirma que un PE Windows tenga los mismos bytes que un ELF Linux. Los manifiestos conservan cada digest junto a la identidad del artefacto.

Stage0 permanece como oráculo de AST en el corpus de esa validación y no como constructor de B y C. Esto es un avance verificable hacia la construcción propia. El dossier mantiene explícitamente abierto Stage3 global y conserva la toolchain instalada.

Las cifras del dossier describen sus corridas: 147 programas completos y 1472 checks por corrida, 38 contratos de runtime por generación y sistema, y comprobaciones de layout en diez perfiles. Esta publicación cita esos resultados históricos; sus smoke tests de entrega se registran aparte y no sustituyen la reproducción completa.

El soporte de un perfil se amplía con evidencia de las rutas pendientes: entradas, artefactos emitidos, dependencias y ejecución. La compatibilidad declarada debe corresponder al alcance de esas comprobaciones.

Reproducir y revisar evidencia

`bootstrap/frontend-v81-build.json` fija las fuentes de la reconstrucción. Los constructores comprueban SHA-256 al copiar y rechazan destinos que contienen `.ncc`. Esa condición evita confundir una generación limpia con la reutilización de objetos previos.

La secuencia conceptual es construir B con la semilla, construir el runtime con B, construir C con B y sus proveedores, y repetir los controles sobre C. Los comandos completos y argumentos de cada constructor se documentan en `docs/NCC_MODULES_V81_REPRO.md`.

```text
semilla -> compilador B -> runtime B
compilador B + runtime B -> compilador C -> runtime C
comparación de artefactos y ejecución de contratos
```

Los informes deben conservar el exit code y la salida esperados, los objetos utilizados y las diferencias exactas. V81 conserva intentos fallidos separados de las corridas finales; un reintento exitoso no debe borrar la explicación del fallo previo.

Una prueba adversa demuestra que el verificador puede rechazar entradas alteradas. Los controles descritos incluyen materiales con alineaciones no válidas, offsets incoherentes e imágenes modificadas. Una imagen cuyo control falla no debe ejecutarse para intentar compensar el rechazo.

Los ensayos de unwind documentados aplican APIs reales de Windows sobre metadatos y un contexto de prueba. No equivalen a provocar una excepción real en cada instrucción del prólogo. Esa limitación pertenece a la afirmación técnica y debe permanecer visible.

Las recetas de reproducción pueden requerir herramientas del sistema, reservas de pila y runners externos. Registra esas condiciones antes de comparar un resultado propio con el dossier.

Publicación y contribuciones

Una publicación útil ofrece un camino de entrada pequeño y comprobable. El lector debe encontrar el propósito del lenguaje, los requisitos, un ejemplo, el resultado esperado y un lugar para informar de problemas. Los paquetes deben identificar versión, destino, hashes y runtime incluido.

El repositorio de desarrollo contiene material que no pertenece a una distribución: cachés, informes locales, archivos temporales y artefactos de construcción. La preparación pública usa una selección explícita de archivos y conserva un manifiesto de hashes. Los binarios se entregan como activos de versión para mantener liviano el árbol fuente.

Una versión de bootstrap puede publicarse como preliminar con límites claros. No debe tener insignias de pruebas que no se han ejecutado, una descarga que apunte a un archivo inexistente ni comparativas de rendimiento sin medición reproducible.

La contribución técnica debe describir el problema, el contrato que cambia, el comportamiento anterior y posterior y la validación realizada. Si toca parser, AST, semántica o IR, explica su continuidad. Si toca ABI o memoria, incluye límites y casos negativos. Si solo mejora documentación, comprueba sus enlaces y ejemplos.

Fuentes técnicas

- `AGENTS.md` y `docs/syntax.md`.
- `crates/ncc-parser/src/lib.rs` y `crates/ncc-stage0-bridge/build.rs`.
- `docs/NCC_MODULES_V81_DOSSIER.md`.
- `docs/NCC_MODULES_V81_ABI_LAYOUT.md` y `docs/NCC_MODULES_V81_REPRO.md`.
- `docs/NCC_MODULES_V81_ATTEMPTS.md`.
- `bootstrap/frontend-v81-build.json` y `bootstrap/frontend-v81-contracts.json`.

Estas referencias fijan el alcance de esta edición. La evolución posterior debe actualizar los contratos, sus pruebas y la documentación pública en la misma entrega.